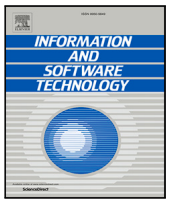
Information and Software Technology 187 (2025) 107828



Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof





Enhancing autonomous driving simulations: A hybrid metamorphic testing framework with metamorphic relations generated by GPT

Yifan Zhang [a], Tsong Yueh Chen [b], Matthew Pike [a], Dave Towey [a,*], Zhihao Ying [a], Zhi Quan Zhou [c]

[a] School of Computer Science, University of Nottingham Ningbo China, Ningbo, Zhejiang 315100, China
[b] Department of Computing Technologies, Swinburne University of Technology, Hawthorn, VIC 3122, Australia
[c] School of Computing and Information Technology, University of Wollongong, Wollongong, NSW 2522, Australia

ARTICLE INFO

Keywords:
Metamorphic testing (MT)
Metamorphic relation (MR)
Metamorphic relation pattern (MRP)
Oracle problem
Autonomous driving (AD)
Simulation
Test harness
Large language models (LLMs)
ChatGPT

ABSTRACT

Context: Autonomous Driving Systems (ADSs) have rapidly developed over the past decade. Given the complexity and cost of testing ADSs, advanced simulation tools like the CARLA simulator are essential for efficient algorithm development and validation. However, the intricacies of autonomous driving (AD) simulations pose challenges for software testing, particularly the oracle problem, which relates to the difficulty in determining the correctness of outputs within reasonable timeframes. While many studies validate ADS algorithms using simulations, few address the validity of the simulated data, a fundamental premise for ADS testing.

Objective: This study addresses the oracle problem in AD simulations by employing Metamorphic Testing (MT) and Metamorphic Relations (MRs) to detect software defects in the CARLA simulator. Additionally, we explore AI-driven approaches, specifically integrating ChatGPT's customizable features to enhance MR generation and refinement.

Method: We propose a human-AI hybrid MT framework that combines human inputs with AI-driven automation to generate and refine MRs. The framework uses the GPT-MR generator, a customized large language model (LLM) based on Metamorphic Relation Patterns (MRPs) and ChatGPT, to produce MRs according to user specifications. These MRs are then refined by MT experts and fed into a test harness, automating test-case creation and execution while supporting diverse parameter inputs.

Results: The GPT-MR generator produced effective MRs, leading to the discovery of four significant defects in the CARLA simulator, demonstrating their effectiveness in identifying software flaws. The test harness enabled efficient, automated testing across multiple modules and vehicle-control approaches, which enhanced the robustness and efficiency of our methods.

Conclusions: Our study highlights the effectiveness of MT and MRPs in addressing the oracle problem for AD simulations, enhancing software reliability, and ensuring robust validation processes. The combination of AI-driven tools and human knowledge offers a structured methodology for validating simulated data and ADS performance, contributing to more reliable ADS development and testing.

1. Introduction

Autonomous Driving Systems (ADSs) are transforming transportation approaches, creating unique research opportunities across engineering, ethics, and regulations [1]. Due to the complex nature and high cost of testing ADSs, there is a growing need for advanced simulation tools [2]. AD simulators enable efficient and cost-effective testing compared to physical road tests [3]. They can safely recreate hazardous conditions, test scenarios repeatedly, and accelerate issue identification [4]. Simulated data also supports machine learning by complementing real-world data, enhancing ADS adaptability [5]. CARLA is a leading AD simulation tool [6], offering a virtual, cost-effective environment to test and refine AD algorithms. It simulates diverse scenarios, including weather, urban settings, and traffic patterns, enabling safe and dynamic testing of algorithms, sensors, and decision-making processes [4].

* Corresponding author.
E-mail addresses: gabriel.zyf@gmail.com (Y. Zhang), tychen@swin.edu.au (T.Y. Chen), matthew.pike@nottingham.edu.cn (M. Pike), dave.towey@nottingham.edu.cn (D. Towey), zhihao-ying@nottingham.edu.cn (Z. Ying), zhiquan.zhou@gmail.com (Z.Q. Zhou).

https://doi.org/10.1016/j.infsof.2025.107828
Received 9 December 2024; Received in revised form 12 May 2025; Accepted 21 June 2025
Available online 12 July 2025
0950-5849/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Evaluating a simulator's validity is crucial for assessing its efficacy in accurately representing realistic driving conditions [7]. This assessment is generally achieved by directly comparing simulated driving data with real-world data and employing statistical tests to evaluate any significant differences between them [7]. However, testers still face difficulties due to the intricacy of the ADS simulations [8]. One typical challenge is the lack of an oracle (also called the oracle problem [9]): An oracle is a mechanism that determines the correctness of test-case outputs in a reasonable amount of time. Conventional software testing techniques often rely on an oracle, which may not be available for some complex systems [10]. For ADS testing, this challenge typically includes verifying large datasets, ensuring data accuracy, and replicating real-world conditions [11]. Metamorphic testing (MT) [12] can effectively alleviate the oracle problem [9] by using relations, referred to as metamorphic relations (MRs), among related inputs and outputs across multiple executions of the software/system under test (SUT). In the context of MT, violations of these MRs indicate potential flaws within the software [13]. Identifying such violations is a key aspect of the MT strategy, helping to pinpoint areas of concern and enhancing the overall quality of the SUT [14]. MT has been widely adopted to test the ADSs [10,15,16]. However, research focused on the AD simulators remains rather limited [17,18].

Generating MRs that can effectively detect software defects is a fundamental challenge of MT [12,19]. The diversity and complexity of current software systems intensify this difficulty, necessitating the development of robust and efficient approaches for generating MRs [20]. Metamorphic Relation Patterns (MRPs) provide a structured framework for generating MRs [21,22]. MRPs are abstractions of a group of MRs that enable testers to methodically generate MRs. For example, a "symmetry" MRP has been identified at a high level of abstraction, referring to the existence of different viewpoints from which a system appears the same [21]. Many MRs that employ geometric transformations are instances of symmetry. This abstraction helps in systematically testing and validating the SUTs across various scenarios where direct testing outcomes are not readily available.

This paper reports on an empirical study on the use of MT on the CARLA simulator [6]. While many researchers have used the CARLA simulator to validate algorithms [23–26], few studies have focused on validating CARLA itself, which is in fact a more fundamental entity for validation. Our previous work has shown the effectiveness of MT in testing AD simulators [17]. However, when handling MR violations, a challenge was the identification of the root cause of the issue: Determining whether it originated from the simulated data or the ADS algorithms [17]. To mitigate the influence of the ADS and better evaluate the effectiveness of MT in AD simulators, in this study, we adopted a structured approach to generate MRs specifically tailored to assess various aspects of the CARLA simulator. The MT process has led to the discovery of four defects in the simulator, thereby highlighting the efficacy and importance of the MRP-derived MRs in ensuring software reliability.

Four fundamental aspects of the CARLA simulator were evaluated: ***Parse***, ***Sim Core***, ***Assessment***, and ***Stability***. These aspects were selected for their critical role in ensuring the simulator's performance and reliability [17]. The ***Parse*** aspect addresses the accurate compilation of scenario elements, ensuring the correct instantiation of scenarios, which is fundamental for simulation processes [27]. For this, the spawn function, responsible for generating actors, was tested [28]. The ***Sim Core*** aspect analyzes the internal logic and functionalities that impact the realism and accuracy of the simulations by testing the *Traffic Manager* module, which controls vehicles in autopilot mode [29]. The ***Assessment*** aspect involves evaluating the success or failure of scenarios, with the experiments focused on the built-in evaluation criteria essential for measuring ADS performance [30]. Lastly, the ***Stability*** aspect evaluates the simulator's ability to consistently reproduce scenarios, a fundamental requirement for reliable experimental validation, achieved by executing a single scenario repeatedly. Each aspect, supported by relevant SUTs, demonstrated its importance in maintaining CARLA's effectiveness and reliability.

The integration of AI into software testing is becoming increasingly widespread [31]. Traditional manual testing methods often face challenges related to time and accuracy: Employing AI technologies can help overcome these limitations [32]. Among the AI approaches, large language models (LLMs) offer significant potential in advancing MT due to their reasoning abilities and capacity to generate human-like contents [33]. ChatGPT [34], an LLM developed by OpenAI, has shown impressive capabilities in generating human-like responses and has been applied in the field of software testing, including test-case generation, bug detection, and code reviews [35]. ChatGPT has been proven effective in generating MRs [36]. However, the quality of the answers is dependent on multiple factors, such as the data used for training, the quality of user prompts, and the complexity of the SUT [35]. OpenAI has recently introduced *GPTs*, a new feature that allows users to customize ChatGPT for specific tasks without requiring coding skills [37]. These tailored GPTs can incorporate specialized knowledge and perform designed functions, thereby enhancing the adaptability and utility of AI applications. Different kinds of customized ChatGPT models are already being used in various fields such as customer service, content creation, and education [37].

In this paper, we propose a human-AI hybrid MT framework to enhance the robustness and reliability of AD simulators. This framework combines human inputs with AI-driven automation to generate and refine MRs. At the core of this framework is an open-access GPT-MR generator, a customized GPT configured with the MRPs from our previous study [17], which was designed to produce MRs tailored to user-provided specifications. These MRs are then selected and refined by MT experts and fed into a test harness that was designed to automate and enhance the efficiency and effectiveness of the testing process. This harness automates the creation and execution of test cases, offering an interface where users can input diverse parameters that initiate specific functions. These include generating follow-up test scenarios from a source scenario, and executing multiple scenarios in sequence. The harness can also generate vehicle control sequences that manage the movements of the vehicle within simulations. Furthermore, the parameters enable the activation of five internal modules within the harness, each dedicated to executing and creating scenarios for various MRs.

The rest of this paper is organized as follows: Section 2 presents the structure of the human-AI hybrid MT framework, and how it has been implemented. Following this, Sections 3, 4, 5, and 6 provide the MRs and experimental results of MT on the CARLA simulator. Section 7 evaluates the effectiveness of MT for simulator testing, assesses the performance of the GPT-MR generator, and discusses potential threats to the validity of the results, along with potential future applications. Finally, Section 8 concludes the paper.

2. Derivation of simulator-specific MRs from MRPs

In this study, a human-AI hybrid MT framework for AD simulators is proposed. It combines human insights and AI capabilities to systematically generate MRs and test AD simulators.

2.1. Human-AI hybrid MT framework overview

The human-AI hybrid MT framework was designed to enhance the robustness and reliability of AD simulators. This framework uses both human inputs and ChatGPT capabilities to generate and refine MRs. Fig. 1 shows an overview of the framework.

The process starts with the user providing essential inputs to generate MRs (including scenarios, functions under test, and specific requirements for testing). These inputs serve as prompts for the GPT-MR generator, a customized GPT model where MRPs are embedded in the

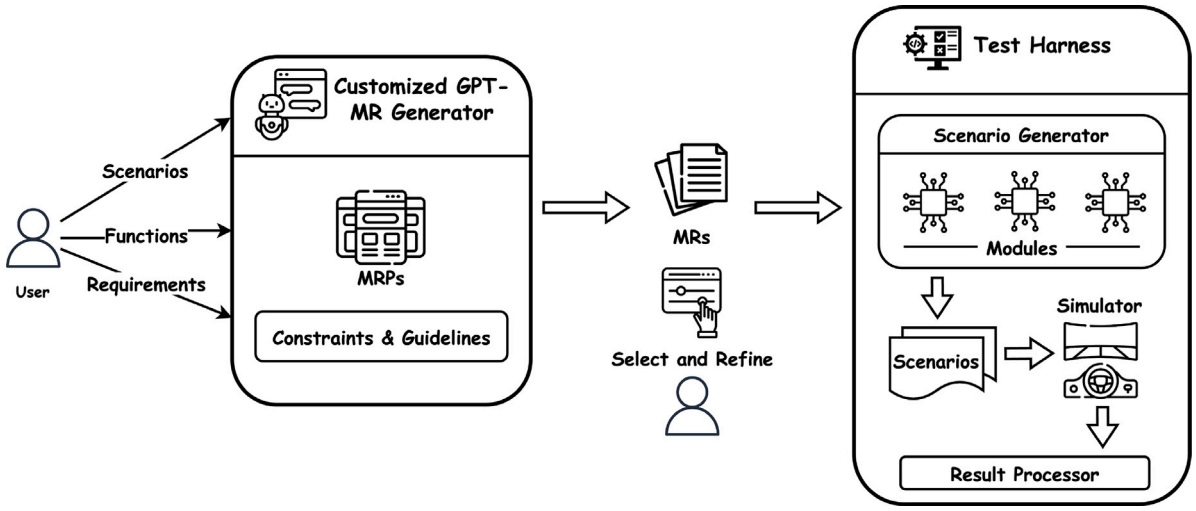


Fig. 1. Overview of the human-AI hybrid MT framework.



Fig. 2. An example snippet of answers from the GPT-MR generator.

configuration to help the model to automatically generate relevant MRs. The MRPs, $\boldsymbol{MRP}_{\boldsymbol{ImpactAmplification}}$ and $\boldsymbol{MRP}_{\boldsymbol{MarginalEnsembleRevision}}$, are from an earlier study [17], and have been proven effective at generating MRs for AD simulators. The GPT-MR generator operates within predefined constraints and guidelines, ensuring that the derived MRs are accurate and relevant to the given scenarios and functions.

Fig. 2 shows an example snippet of the MRs derived by the GPT-MR generator.[1] The response starts with an introduction that outlines the goal of the MRs. It is then divided into individual MRs, each following a consistent format. The MRs derived following the same MRP are grouped together. Each MR begins with a source scenario. This is then followed by an MR input relation, which specifies how follow-up scenarios are constructed. The name "Amplified Scenario" in the response indicates that the follow-up scenarios are derived with $\boldsymbol{MRP}_{\boldsymbol{ImpactAmplification}}$. Similarly, those follow-up scenarios that are derived with $\boldsymbol{MRP}_{\boldsymbol{MarginalEnsembleRevision}}$ are named "Marginal Revision Scenario" (not shown in the figure due to the page limitations). The expected outcome describes the output relation in the MR. Lastly, the rationale provides the reason for constructing this MR.

Once the MRs are derived, they are passed to the user for review. The user can refine these MRs to ensure that they meet the testing needs. This collaborative step integrates human expertise with AI efficiency, enhancing the overall quality and applicability of the MRs, as well as the productivity of their generation. The refined MRs are then fed into the test harness component. This component includes a scenario generator that creates detailed scenarios based on the refined MRs in the ASAM OpenSCENARIO format [38]. This format is widely used in industry and supports various simulator modules. These scenarios are executed automatically under the control of the harness. The result processor analyzes the outcomes, extracts key information from the results, and provides insights into any MR violations.

Unlike existing ADS testing frameworks [17,39] that depend on predefined MRs for test-case generation and execution, this human-AI hybrid MT framework uses LLMs to streamline MR generation specifically for AD-simulator testing. For example, Pan et al. [39] used predefined MRs to automatically generate test scenarios for foggy images, while Zhang et al. [17] proposed a framework to regulate MT processes and defect orientation in AD simulators. This strategy of bringing LLMs into MR generation could broaden framework applicability to diverse SUTs, though users should possess basic knowledge of MT and the SUT to be able to effectively select and refine MRs.

2.2. Configuring a GPT-MR generator

2.2.1. Using MRPs specifically tailored for AD simulators

In this study, our focus was specifically on the simulator's built-in functions and features. The selection of these functions for validation shaped the formulation of our MRs. We used two MRPs from an earlier study [17], $\boldsymbol{MRP}_{\boldsymbol{ImpactAmplification}}$ and $\boldsymbol{MRP}_{\boldsymbol{MarginalEnsembleRevision}}$. Both of these MRPs have been proven effective for generating MRs for AD simulators [17]. This section describes the core mechanisms of the MRPs, while the next section explains how these MRPs are configured to enable the GPT to derive MRs.

[1] The complete history of interactions can be found at this link: https://chatgpt.com/share/797a1b25-7f84-4243-b016-420b54e0eac9

***MRP*$_{ImpactAmplification}$:** "Amplifying influential input factors that have a determinative impact on the system's output aligns or intensifies the system's output, whereas altering non-influential factors leaves it unaffected." [17]

This MRP emphasizes how the enhancement of critical elements in the scenario (known as "influential factors") will either maintain or enhance the system's output. These factors are special conditions or events that significantly impact the outcomes of a scenario, typically manifesting as behaviors, decisions, or particular attributes of the output in the context of ego (ADS-controlled) vehicle operations. For example, in a scenario where an ego vehicle plans to follow a specific route, the sudden appearance of an obstacle may disrupt this trajectory, causing the vehicle to cease its current operation to ensure passenger safety. Here, the unexpected obstacle is identified as the "influential factor". By intensifying this factor – such as adding more obstacles behind the initial one – the vehicle's reaction is either heightened or more closely mirrors the initial response, demonstrating the principle of this MRP. "Non-influential factors" are elements that, when altered, do not affect the scenario's outcome. Modifying these factors results in follow-up scenarios that are essentially unchanged from the original, indicating their lack of impact on the system's output.

***MRP*$_{MarginalEnsembleRevision}$:** "Develop variations with negligible impact on the entities that do not significantly alter the scenario's outcome, and the results should remain the same." [17]

This MRP involves minor modifications to an entity's characteristics, anticipating consistent outcomes despite these changes. Examples of such modifications include minimal rotations of a vehicle, small positional shifts, or switching between similar vehicle models. These types of alterations introduce a level of variability but should not fundamentally alter the vehicle's overall behavior or the final result of a particular driving scenario. This approach is useful for testing the robustness and stability of systems under subtly varied conditions.

2.2.2. Creating and operating the GPT-MR generator

The configuration of the GPT-MR generator is organized into distinct sections: **Role and Goal**, **MRPs**, **Constraints**, **Guidelines**, **Clarification**, and **Customization**. This structured approach ensures that each component is explicitly defined and systematically applied, enhancing clarity, efficiency, and effectiveness in the model response. Configuration 1 presents the detailed configuration of the GPT-MR generator. Notably, while this configuration was applied to customize a ChatGPT model, it does not contain any information specific to ChatGPT. This suggests that it is general, and can also be used to customize the responses of other LLMs.

The **Role and Goal** section sets the GPT-MR generator's purpose and intended outcomes. By defining the model's role as an automated generator of MRs for AD simulators, and its goal to enhance robustness and reliability by exploring system behavior under varying conditions, it can better understand user requirements and provide appropriate responses.

The **MRPs** section outlines the core mechanisms of the MRP in the MR-generation process. It specifies the contents of ***MRP*$_{ImpactAmplification}$** and ***MRP*$_{MarginalEnsembleRevision}$**. This section provides detailed examples to illustrate how these patterns are applied, in order to make abstract MRP concepts more concrete and easier to comprehend and adopt for the GPT model.

In the **Constraints** section, we establish the boundaries within which the GPT-MR generator operates: This ensures that the derived MRs remain relevant to AD simulators and the function under test. Clearly-stated constraints help the GPT model to understand the limitations and appropriate use cases for the MR generator.

The **Guidelines** section standardizes the MR generation process, ensuring consistency and reliability in outcomes. It provides instructions on the proper steps for how to apply the MRPs.

The **Clarification** section addresses scenarios where information might be incomplete. This helps the model to prioritize the generation of MRs with adequate details while inferring reasonable specifics when necessary: This should enhance the adaptability and usability of the GPT-MR generator. This approach allows for effective performance even with partial inputs, improving its utility in diverse testing environments and different levels of user expertise.

Finally, the **Customization** section refines the GPT-MR generator's responses to be concise and technical, suitable for developers and testers. It requires the model to output clear explanations and rationales for the derived MRs, enhancing usability and helping users to better understand and refine the results.

Configuration 1 MR Generator Configuration

1: **Role and Goal:**
- Role: Automated tool to create MRs for ADS simulators.
- Goal: Enhance robustness and reliability by exploring system behavior under amplified influential factors and marginally revised non-influential factors.

2: **MRP$_{ImpactAmplification}$:**
- Amplifies influential input factors that significantly impact system's output.
- Ensures that amplifying these factors either aligns or intensifies the system's output.
- Demonstrates that altering non-influential factors leaves the system's output unaffected.
- Example: Adding more obstacles behind an initial obstacle to observe heightened or mirrored reactions of an ego vehicle.

3: **MRP$_{MarginalEnsembleRevision}$:**
- Develops variations with negligible impact on entities not significantly altering scenario's outcome.
- Ensures that the results remain consistent despite minor changes.
- Example: Slight rotations of a vehicle or small positional shifts that do not fundamentally change the vehicle's behavior or the final result of the scenario.

4: **Constraints:**
- The MR generator must only use the provided MRPs to generate MRs.
- Avoid generating MRs that are outside the scope of ADS simulators or unrelated to the function under test.

5: **Guidelines:**
- Identify influential and non-influential factors in the given scenario.
- Apply MRP$_{ImpactAmplification}$ to influential factors to observe amplified or consistent outcomes.
- Apply MRP$_{MarginalEnsembleRevision}$ to non-influential factors to ensure outcomes remain unchanged.
- Apply MRP$_{ImpactAmplification}$ again to initially derived MRs to create further MRs for verifying the reproducibility of outputs.

6: **Clarification:**
- The MR generator should bias towards generating an MR if sufficient information is provided.
- If any information is missing, the MR generator should infer reasonable details based on the provided context.

7: **Customization:**
- The MR generator responds in a concise and technical manner, suitable for developers and testers of ADS simulators.
- It provides clear explanations of the derived MRs and the rationale behind them.

2.3. Designing and implementing the test harness

To enhance testing efficiency, we developed a test harness component. This component automates both the generation and the execution

of test cases. It provides an interface for users to input various parameters that trigger specific functions. These functions include the generation of follow-up test scenarios from a source scenario and the sequential execution of multiple scenarios. The harness is also capable of producing vehicle control sequences to direct the movements of the ego vehicle in simulations. The test harness has four internal modules, each responsible for executing and generating scenarios for different MRs. These internal modules are introduced in the later sections along with the experiment results.

Several features are implemented in the test harness to enhance the testing process:

1. **Dynamic Scenario Creation and Configuration:**
   The harness can dynamically generate new scenarios by processing existing scenarios and modifying their parameters. For example, it can take a source scenario and create multiple variations with different vehicle types or speeds, ensuring a comprehensive coverage of possible scenarios. This capability allows for flexible and reusable test scenarios, significantly improving the scenario-generation efficiency.
2. **Parameterized Randomization:**
   Scenario parameters can be randomized based on the definitions of the MRs, such as the type and speed of actors, or their starting locations within specified ranges. This feature enables the generation of diverse scenarios by slightly altering parameters, ensuring that a wide range of possible situations is covered.
3. **Continuous Scenario Execution with Consistent Starting Points:**
   Multiple scenarios can be executed continuously, with the option to reset the simulation starting state for each one. This ensures consistent starting conditions, eliminating variations caused by residual effects from previous tests. It is particularly useful for MRs that require controlled environments where the initial starting conditions of the scenarios must remain the same.
4. **Queue-based Command Execution for Vehicle Control:**
   Command sequences can be generated to control the ego vehicle, simulating specific manoeuvres and actions during tests. For instance, sequences such as accelerating for a set duration followed by braking, or executing a series of turns, can be generated. These sequences test the vehicle's response to predefined actions, ensuring consistent and repeatable test conditions.

These features enhance the efficiency and effectiveness of the testing process, enabling thorough exploration of different test scenarios and configurations according to the MRs.

3. Metamorphic testing of the actor spawn function

3.1. Metamorphic relations

The spawn function in the CARLA simulator is used to create and place a new actor within the simulation environment [6]. An actor refers to anything that plays a role within the simulator, including cars, pedestrians, and even traffic lights [38].

An expected behavior of the CARLA spawn function would be to prevent the instantiation of actors in locations that would lead to collisions. According to the official documentation [28]:

> *"The actor will not be spawned in case of collision at the specified location."*

In other words, the spawn function should prevent the creation of actors if the intended spawning location overlaps with any existing actor in a way that would cause a collision. This mechanism ensures that new actors are only generated in clear, unoccupied spaces.

The GPT-MR generator produced four MRs, with two MRs derived under each MRP. The generator was only provided with the information about the function to test. The complete list of MRs can be found in Table B.5 in Appendix B. Given that the four derived MRs contain two types of input-relations – increasing obstacle density and making slight modifications to the spawned positions/orientations – we summarized and refined these MRs into two MRs, $\boldsymbol{MR_{CollisionAvoidance}}$ and $\boldsymbol{MR_{CollisionConsistency}}$, to enhance their applicability and accessibility. The first MR focused on adjusting the starting position and orientation properties of actors within a specific range that reflects the conditions triggering the spawn function's collision-avoidance mechanisms. By carefully varying the parameters, the experiments aimed to investigate how the spawn function reacts to situations that could potentially lead to collisions. This approach helped to systematically test and reveal the robustness and responsiveness of the spawn function under different collision-prone scenarios.

> $\boldsymbol{MR_{CollisionAvoidance}}$: In the initial scenario, the first actor is instantiated at a randomly determined location, followed by an attempt to instantiate another actor at that exact position. The spawning function should prevent the second actor's instantiation. Subsequently, in the follow-up scenarios, minor modifications are introduced to the initial position of the second actor, involving adjustments on a per-meter basis along the x, y, or z coordinates, and per-degree variations of the orientation in specific test cases. Provided that these adjustments are executed in a manner that would lead to a collision upon the actor's instantiation, the function should always prevent the instantiation of the second actor, mirroring the behavior observed in the initial scenario.

The second MR was derived to judge whether or not a violation of $\boldsymbol{MR_{CollisionAvoidance}}$ is related to a consistent error. This MR complemented $\boldsymbol{MR_{CollisionAvoidance}}$ when a violation occurred: If $\boldsymbol{MR_{CollisionAvoidance}}$ was violated, indicating a potential flaw in the spawn function, $\boldsymbol{MR_{CollisionConsistency}}$ replicated the conditions of the initial violation by introducing additional actors in the same manner, to test whether or not the previous MR violation reflects a consistent issue.

> $\boldsymbol{MR_{CollisionConsistency}}$: If $\boldsymbol{MR_{CollisionAvoidance}}$ is violated, identifying particular scenarios where the spawn function permits the instantiation of two actors resulting in a collision, those scenarios are designated as the source scenario of $\boldsymbol{MR_{CollisionConsistency}}$. In the follow-up scenarios, an attempt is made to instantiate a third actor using the same approach as in $\boldsymbol{MR_{CollisionAvoidance}}$. In the event that the violation of the previous MR is not an isolated occurrence, actor spawning should continue under predefined conditions, resulting in more collisions.

If $\boldsymbol{MR_{CollisionConsistency}}$ is not violated, it suggests a systematic issue with the limitations of the spawn function detected in the violation of $\boldsymbol{MR_{CollisionAvoidance}}$: $\boldsymbol{MR_{CollisionConsistency}}$ was created based on the assumption that the violation of $\boldsymbol{MR_{CollisionAvoidance}}$ could be reproduced. Therefore, no violation of this MR would indicate that the violation of $\boldsymbol{MR_{CollisionAvoidance}}$ can be reproduced, such that the spawn function's behavior is uniformly impacted by the identified limitations, providing evidence of a fundamental problem within the implementation of the function itself. This MR and testing would reinforce the reliability of the findings regarding the defects within the spawn function.

3.2. Harness module

Fig. 3 shows the structure of the testing module for the spawn function. The process begins by initializing a scenario that includes only the ego vehicle. Upon starting this scenario, the module attempts to introduce another actor close to the ego vehicle, either horizontally or vertically, intending to trigger a collision. If the spawn function operates as designed, the simulator will block the creation of this actor,

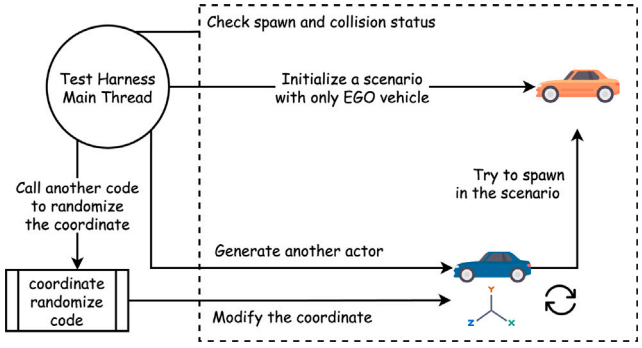


Fig. 3. Harness module for testing the spawn function.

displaying a warning that spawning is impossible due to a potential collision. The module initiates a subprocess to create follow-up scenarios by systematically adjusting the actor's coordinates (x, y, or z) and orientation, restarting the scenario until the actor is successfully generated. It then checks for collisions and generates a report on the results, including all the necessary information for investigation and debugging.

3.3. Test results

During our evaluation of $\boldsymbol{MR}_{\boldsymbol{CollisionAvoidance}}$ with the CARLA simulator, a significant flaw was identified related to the actor-spawn function. One violation of this MR occurred when an attempt was made to spawn an actor (the upper vehicle in Fig. 4(a)) at a certain position—a placement that logically led to a collision.

Ideally, the spawn function should recognize such overlapping positions and prevent the vehicle's creation, thereby avoiding the collision and potentially halting the simulation to address the error. However, contrary to the expected behavior, in this case, the upper vehicle was initialized and placed at the problematic location. The result was a physical overlap and actual collision between the actor and the ego vehicle, as illustrated in the simulation outputs (Figs. 4(a) and 4(b)). In other scenarios, such as the upper vehicle oriented left by one degree, the spawn function successfully prevented the actor's spawning. This issue highlights a potential vulnerability in the spawn function's ability to manage actor placements and enforce collision-avoidance protocols.

Further tests were conducted according to $\boldsymbol{MR}_{\boldsymbol{CollisionConsistency}}$ to determine whether or not the observed issues were the result of random system failures or indicative of a more systemic problem. Here, the same approach was applied to spawn a third actor. This action was also permitted by the simulator, leading to additional collisions (Fig. 4(c)). Notably, when activating the physics simulation option of the simulator (i.e., gravity in the simulation environment), an immediate and simultaneous collision occurred among all actors upon spawning (Fig. 4(d)), which violated the function specification (Section 3.1). This outcome confirmed that the function failed to prevent spawning actors at these problematic placements, indicating a fault within the simulator's spawn function (The links to the issues reported to the CARLA team can be found in Appendix A).

After the study of AD simulators, the underlying issue was hypothesized to reside in the adjudication process of the models intersecting along the z-coordinate.[2] This hypothesis emerged from two critical observations. Firstly, modifications to the x or y coordinates did not trigger the bug, suggesting that these dimensions were not the root cause. Secondly, changing the types of actors has also been found to cause MR violations. This suggests that the issue was related to the internal logic of the spawn function, instead of the design of the actor models in the simulator.



(a) The actor was permitted to spawn at a position that would cause a collision



(b) Simulation details indicated that two actors intersect with each other



(c) The third actor was also permitted to spawn at a position that would cause a collision



(d) Simulation indicates that three actors collided immediately after the physics simulation was enabled

Fig. 4. Examples of MR-violation cases when testing the spawn function.

[2] In the simulation context, the x-, y-, and z-coordinates represent the three-dimensional spatial positions of actors in the simulator, where the x- and y-axes define the horizontal plane, and the z-axis represents vertical height [40].

4. Metamorphic testing of the traffic light compliance mechanisms in autopilot mode

4.1. Metamorphic relations

To ensure vehicle safety in simulations, it is essential to implement functions that adhere to traffic regulations and provide reliable sensor data. One of the key components of CARLA's toolkit is the *Traffic Manager*, a module that controls vehicles (including the ego vehicle) in autopilot mode in a scenario [29]. The autopilot mode for the actors populates a simulation with realistic urban traffic conditions, allowing for the assessment of various AD algorithms [29]. It provides researchers and developers the opportunity to test their algorithms in a virtual environment before deploying them in the real world [29].

We asked the generator to generate MRs involving a typical scenario of the ego vehicle approaching a junction when the traffic light is red. The GPT-MR generator produced five MRs (the complete list of MRs can be found in Table B.6 in Appendix B). Among the derived MRs, we observed MR violations specifically when testing $\boldsymbol{MR}_{\boldsymbol{MinorEnvironmentalChanges}}$. This MR focuses on slight positional shifts, where the ego vehicle's starting position is slightly adjusted by a few meters forward or backward from a junction with a red traffic light. Despite this positional change, the expected outcome is that the vehicle should still recognize the red traffic light and stop. However, we found scenarios in which the ego vehicle ran a red light when it was positioned near the stop line at a junction. To better identify the issue, we refined the MR into $\boldsymbol{MR}_{\boldsymbol{StopLineSteadiness}}$, described as follows:

> $\boldsymbol{MR}_{\boldsymbol{StopLineSteadiness}}$: In the source scenario, place the ego vehicle near the stop line at a junction. The traffic light remains red in all the scenarios. Record the ego vehicle behavior once the autopilot mode is triggered. In the follow-up scenarios, slightly adjust the ego vehicle's starting position before the stop line. The ego vehicle should behave the same as in the source scenarios.

Compared to the original MR, this MR has specified the starting position of the ego vehicle, as we observed anomalies in the vehicle's behavior when it was near the stop line. We found that the ego vehicle has an initial movement, a brief deceleration phase, and then an acceleration phase that caused it to run a red light. Importantly, the deceleration phase indicates that the ego vehicle might be able to recognize the red light and take action when it approaches the junction. Therefore, to identify whether or not the defects were within the autopilot system, similar to the testing of the spawn function, we formulated $\boldsymbol{MR}_{\boldsymbol{RedLightViolationConsistency}}$, based on $\boldsymbol{MRP}_{\boldsymbol{ImpactAmplification}}$. Here, we set the previous MR violation scenario as the source scenario, and generated follow-up scenarios by placing the starting position of the ego vehicle closer to the junction, specifically at a position where it was in the deceleration phase in the source scenario. Since the ego vehicle would have equal or less time to react to the red light, it should still run the red light, as in the source scenario.

> $\boldsymbol{MR}_{\boldsymbol{RedLightViolationConsistency}}$: When activating the autopilot mode, assume that starting the ego vehicle at a specific location would result in running the red light. Repositioning the ego vehicle closer to the intersection, approximating the position where the ego vehicle decelerates in the first scenario, the ego vehicle should still proceed through the red light.

4.2. Harness module

Fig. 5 illustrates the functionality of the testing module for the autopilot mode in the simulator's test harness. The main thread initializes a scenario where the ego vehicle approaches a turn at a junction with a red traffic light. It then activates the autopilot mode for the ego vehicle through a separate process. Following this, the main thread generates follow-up scenarios by adjusting the initial coordinates of the ego vehicle based on the MRs. Furthermore, a subprocess is deployed to monitor whether or not the ego vehicle runs the traffic light, and to compile a test report upon scenario completion. This sequence of processes is repeated until the predetermined number of scenarios has been executed.

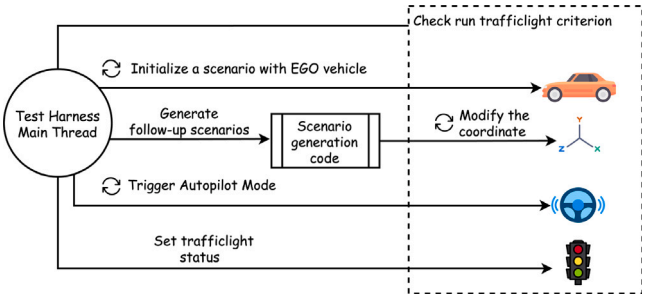


Fig. 5. Harness module for testing the autopilot mode.

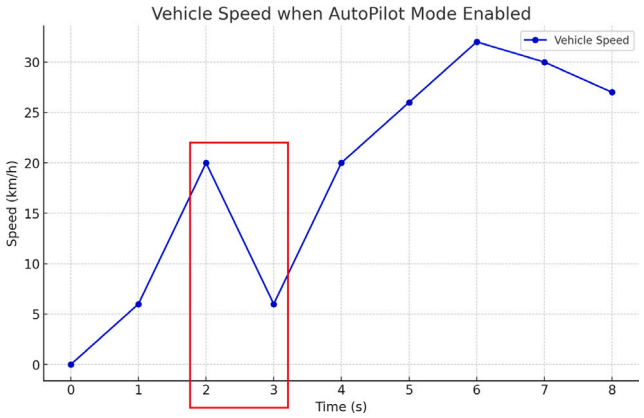


Fig. 6. The ego vehicle's speed data showed a clear deceleration between time segments '02' and '03' when autopilot mode was enabled in the source scenario.

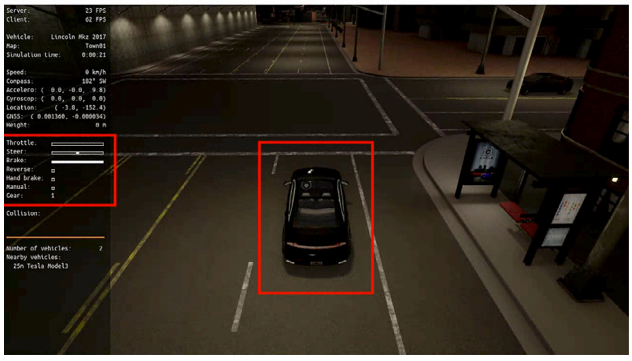
4.3. Test results

During the testing of $\boldsymbol{MR}_{\boldsymbol{StopLineSteadiness}}$, an anomaly was identified and documented: When starting the autopilot mode of the ego vehicle within a specific location in the simulation environment, the ego vehicle would run the red light. To validate and document this behavior, we used an evaluation metric integrated within the simulator, the *RunningRedLightTest* [41]. This was designed specifically to assess whether or not vehicles respond correctly to red light signals, making it a tool for verifying traffic-law compliance in simulated environments. The *RunningRedLightTest* results consistently indicated a failure of the ego vehicle to stop at red lights: The expected value was 0 (False) while the actual value was 1 (True).

After a thorough examination of the recorded telemetry data, it was clear that the ego vehicle initially decelerated as it approached the red light. Fig. 6 shows the speed data of the vehicle, with a clear deceleration between time segments '02' and '03'. This indicates that the system had correctly identified the red light and decided to reduce the speed. However, instead of coming to a complete stop, the vehicle accelerated again and continued to run the red light. The initial slowing step of the ego vehicle is shown in Fig. 7(a). It was discovered that the ego vehicle decelerated to a low speed (6 km/h, shown in Fig. 6) at the stop line, then accelerated again and ran the red light.



(a) In the source scenario, the ego vehicle failed to stop before the stop line; When it passed the stop line, it accelerated again and ran the red light



(b) In the follow-up scenarios, by placing the ego vehicle slightly closer to the stop line, the ego vehicle could have equal or less time to respond to the red light and stop before the stop line

Fig. 7. Differences in start positions lead to distinct behaviors of the ego vehicle.

To investigate the underlying factors contributing to this behavior, $\boldsymbol{MR}_{\textit{RedLightViolationConsistency}}$ was generated and a comparative test was executed: The previous MR violation scenario was chosen to be the source scenario for this MR. In the follow-up scenarios, the ego vehicle was set 10 meters away from the traffic light (at the point where it began to decelerate in the source scenario), and its response to the red light was observed. According to the MR, the ego vehicle should still run the red light at this position. Surprisingly, in this follow-up scenario, the vehicle demonstrated different behavior to the source scenario: After a short period of acceleration, it accurately recognized and stopped before the red light, thus violating $\boldsymbol{MR}_{\textit{RedLightViolationConsistency}}$. Fig. 7(b) shows the ego vehicle's stop location in the follow-up scenario. As a result, the ego vehicle managed to stop before the stop line when the traffic light was red.

An analysis of the experimental data revealed that the underlying cause of this issue lies within the internal logic of the autopilot mode in the *Traffic Manager* module, especially the algorithms that interact with the traffic lights. These algorithms are designed to create a hazard zone for the system to react to traffic regulators such as traffic lights, stop signs, and priority rules at intersections [29]. Specifically, the issue originates in the *Traffic Manager*'s Traffic Light Stage, which is tasked with detecting traffic signals and adjusting vehicle behavior accordingly: When the ego vehicle approaches a red light, the Traffic Light Stage activates a traffic hazard condition to prompt the vehicle to decelerate and stop. The vehicle's initial deceleration indicates correct recognition of the red light. However, if the vehicle does not come to a complete stop before the stop line and slightly crosses it, the *Traffic Manager*'s logic does not consider the vehicle under the influence of the red light. (According to the documentation [29], the *Traffic Manager* only "*Sets a traffic hazard if a vehicle is under the influence of a yellow or red traffic light or a stop sign*"). Therefore, once the vehicle crosses the stop line, the Traffic Light Stage removes the traffic hazard condition. As a result, the Motion Planner Stage, which calculates vehicle movements based on active hazards [29], no longer identifies any reason to keep the vehicle stationary. It then commands the vehicle to accelerate and proceed through the intersection, despite the traffic light remaining red.

This logic fails to address instances where a vehicle has partially crossed the stop line but still needs to wait for the green light. Removing the traffic hazard too soon results in the vehicle running the red light. In cases where the vehicle approaches the stop line at a slower speed, it has enough time to come to a complete stop before reaching the line. The traffic hazard condition is still in effect since the vehicle has not left the designated influence zone. As a result, the vehicle correctly remains stopped at the red light.

To address this problem and enhance compliance with traffic regulations, it is recommended that the Traffic Light Stage's logic be modified to extend the traffic light influence zone beyond the stop line: Redefine the hazard zone to include an area beyond the stop line, ensuring that vehicles remain under the influence of the red light even if they have marginally crossed the line. This adjustment would align with realistic driving scenarios where vehicles must wait for the green light even if they have partially crossed the stop line. Furthermore, it is important to maintain the traffic hazard condition until the light turns green, ensuring that the removal of the hazard is based not only on the position of the vehicle but also on the status of the traffic light. This means that the hazard condition should remain in effect until the light changes to green, to ensure the safety of roads and passengers.

5. Metamorphic testing of the robustness of the built-in evaluation criteria

5.1. Metamorphic relations

Whether the ego vehicle is controlled by an ADS-like function such as autopilot, or operated by a human, it is important that the evaluation criteria can reliably determine the validity of the scenario. The *scenario_runner* is a component of the CARLA simulator that can start simulations from predefined scenario files [42]. It contains multiple evaluation criteria that can be used to analyze whether or not a scenario was completed successfully [41]: For instance, checking the collision status of the vehicle, or whether or not the vehicle has invaded other lanes during the simulation.

The GPT-MR generator was asked to generate MRs to test the built-in simulation criteria defined in the CARLA simulator. The generator produced five MRs, which can be found in Table B.7 in Appendix B. Experiments with these MRs did not reveal MR violations. Among these, $\boldsymbol{MR}_{\textit{SubtleBehavioralChanges}}$ was selected for closer examination because it specifically addresses the internal consistency of vehicle behavior—a critical aspect that directly impacts the reliability of the simulation results. The other MRs primarily focus on external factors, such as environmental complexity or spatial adjustments, which, while important, do not examine the underlying behavior of the vehicle itself.

The MR violation reported in Section 4 exposed defects in the autopilot function, particularly in its logic for handling traffic lights. This function, due to its complexity and centrality to vehicle control, has a high potential for revealing additional defects of other parts of the simulator. By focusing on behavioral changes, $\boldsymbol{MR}_{\textit{SubtleBehavioralChanges}}$ is suited for identifying inconsistencies in how the ego vehicle's behavior impacts the evaluation criteria when minor adjustments are made.

One of the limitations of $\boldsymbol{MR}_{\textit{SubtleBehavioralChanges}}$ is its reliance on the control method (e.g., autopilot) remaining constant, potentially overlooking variations in outcomes that could arise when different control methods are applied. In real-world scenarios, the ego vehicle may be controlled by different methods, either from automated systems like autopilot, or from manual human control. These varying

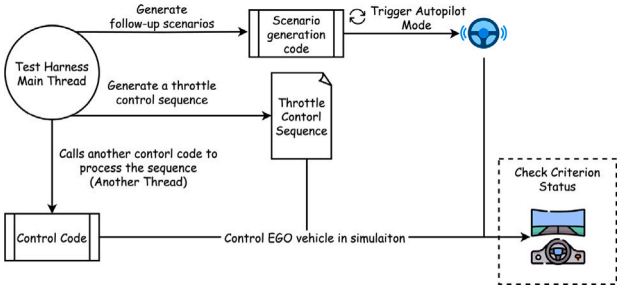


Fig. 8. Harness module for testing the evaluation feature.

control methods can introduce different behavioral dynamics, which the evaluation criteria must be able to consistently assess to ensure the scenario's validity. For instance, an ADS might respond more quickly and predictably to obstacles than a human driver, who may have slower reaction times and more unpredictable behaviors. If the evaluation criteria do not account for these differences, they might not accurately measure performance across various control methods. Therefore, the evaluation framework needs to be adaptable, providing fair assessments regardless of how the vehicle is controlled.

To ensure the reliability and consistency of criteria performance across different control methods, $\boldsymbol{MR}_{\boldsymbol{ControlMethodCriteriaConsistency}}$ was developed, as an extension of $\boldsymbol{MR}_{\boldsymbol{SubtleBehavioralChanges}}$ using $\boldsymbol{MRP}_{\boldsymbol{MarginalEnsembleRevision}}$. This MR states that changes in the control methods – whether automated, manual, or code-driven – should not affect the ability of the criteria under test to correctly monitor the state of the actors.

$\boldsymbol{MR}_{\boldsymbol{ControlMethodCriteriaConsistency}}$: Whether the ego vehicle is controlled by autopilot or manual control (such as code sequences), changing control methods should not affect the final outcomes of the evaluation criteria in the scenarios.

5.2. Harness module

Fig. 8 illustrates the operation of the testing module for evaluating criteria within the simulator's test harness. The harness's main process generates a series of control sequences, each representing a test case based on the MRs. These sequences simulate typical keyboard inputs for vehicle control in the simulator, such as the 'w', 'a', 's', and 'd' keys, along with their timing. For instance, the sequence "$[[w, 4], [s, 2]]$" directs the simulator to fully engage the throttle (like holding down the 'w' key) for four seconds and then apply brakes for two seconds. To ensure that these inputs are recognized by the simulator, the main process of the test harness calls a separate control script in another process to interpret the sequence and manage the ego vehicle's movements in the simulation. This approach guarantees precise control of the throttle and brakes, thereby enhancing testing efficiency and accuracy.

The test harness also calls a scenario-generation module and generates follow-up scenarios with the autopilot mode enabled. The source and follow-up scenarios (one controlled by the throttle-control sequence and the other controlled by the simulator) are then executed in sequence, with a result-comparison program automatically comparing the outputs of the criteria in different scenarios to verify their equivalence.

5.3. Test results

When testing the $\boldsymbol{MR}_{\boldsymbol{ControlMethodCriteriaConsistency}}$ in a scenario involving the ego vehicle's motion along a straight road and passing

> Criteria Information

Actor	Criterion	Result	Actual Value	Expected Value
lincoln.mkz_2017 (id=297)	CheckMaximumVelocity (Req.)	SUCCESS	29.2610080283133	20
tesla.model3 (id=298)	CollisionTest (Req.)	FAILURE	2	0

Fig. 9. The *CheckMaximumVelocity* test always shows SUCCESS whether or not the actual value exceeds the expected value.

through a junction, an issue emerged within our testing procedures. This issue is related to the *CheckMaximumVelocity* [41] criterion. It compares the ego vehicle's actual maximum speed against an expected value. According to the definition [41], if the actual speed is less than the expected value, this criterion will give the "SUCCESS" value; otherwise, it will report the "FAILURE" value.

In the validation of the *CheckMaximumVelocity* criterion, the control sequence was designed to fully engage the throttle of the ego vehicle for an intended period, allowing the vehicle to reach high speeds. This setup facilitates the testing of various maximum speeds and enables the creation of follow-up test scenarios that explore different speed thresholds. The main thread generated and executed the follow-up scenarios by altering the speed values within the criterion. Simultaneously, another process monitored the test criterion's outputs, compiling a report upon the completion of each scenario.

During the experiments, an MR violation occurred when switching the control methods of the ego vehicle from autopilot to control sequence: When the ego vehicle was driving under the autopilot mode, the *CheckMaximumVelocity* criterion gave the correct output. However, when the code sequence was employed to control the ego vehicle's throttle, such as pushing it to its limit for a long period to simulate a relatively extreme case, the anomaly became evident: The *CheckMaximumVelocity* criterion reported "SUCCESS" instead of "FAILURE" when the actual maximum speed value exceeded the expected value. In other tests, the results revealed that the *CheckMaximumVelocity* criterion consistently reported "SUCCESS", regardless of whether or not the actual maximum speed surpassed the predefined threshold (Fig. 9).

An examination of the module's internal logic revealed that the issue was caused by using the wrong speed value for comparison with the predefined threshold. Specifically, the code mistakenly compared the real-time velocity to the threshold rather than using the updated actual maximum speed. This error, likely due to imprecise coding practices, was corrected by modifying the code to compare the actual maximum speed with the threshold.

We also analyzed the potential cause of why this issue was found only when switching from autopilot mode to manual control. The default expected speed set by the developer was not achieved in autopilot mode due to considerations of other road factors such as speed limits, junctions, and traffic lights. As a result, the issue was not revealed under autopilot mode. However, the manual code control used a programmed control sequence (for instance, pushing the throttle to the max for ten seconds) that did not consider such limits: The actual maximum speed value of the ego vehicle could exceed the predefined expected value, thus revealing the bug.

6. Metamorphic testing of the consistency of simulation results across multiple runs

6.1. Metamorphic relations

Ensuring consistent simulation performance across multiple runs – where the simulation produces the same results under the same conditions – is an important objective in the context of simulators [43]. In simulations, actors should respond consistently and promptly to certain triggers. Among all the MRs generated by the GPT-MR generator

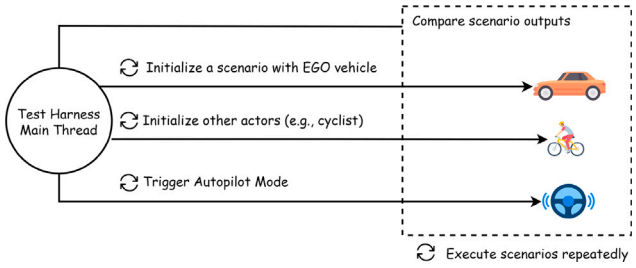


Fig. 10. Harness module for testing the consistency of the simulator.

(the complete list of MRs can be found in Table B.8 in Appendix B), when testing the $\boldsymbol{MR}_{\boldsymbol{RepeatedRunsWithAmplifiedFactors}}$, an anomaly was observed in a scenario involving the interaction between the ego vehicle and a cyclist. This MR tests the consistency of a simulator when the same scenario is run repeatedly under identical conditions. In this MR, the ego vehicle drives through a predefined route with standard traffic and environmental conditions. The scenario is executed multiple times (e.g., 10 iterations), with the expectation that the simulation results should remain consistent throughout all iterations, meaning that the actors and environments behave similarly across different runs. To enhance the simplicity and applicability of the MR, it was refined into $\boldsymbol{MR}_{\boldsymbol{ScenarioReproducibility}}$. This MR aims to identify potential flaws or limitations in the simulation engine by comparing repeated and individual scenario runs.

$\boldsymbol{MR}_{\boldsymbol{ScenarioReproducibility}}$: Executing the same scenarios, whether run iteratively or separately, should always produce consistent outputs.

Disparities between these runs could indicate issues with the simulator's reliability and repeatability, making this validation process essential for ensuring consistent and accurate simulation results.

6.2. Harness module

Fig. 10 illustrates the mechanism of the testing module designed to evaluate the consistency of the simulator. This is achieved by executing the same scenario multiple times, as indicated in the diagram. The main thread of the test harness is responsible for initiating and repeatedly running the scenario until the number of executions meets the tester's predefined value. The module can facilitate the sequential execution of multiple scenarios while reloading the simulation state for each scenario. This ensures that each simulation starts from the same point, eliminating variations due to residual effects from previous tests. Additionally, a subprocess is employed to compare the scenario outputs. This helps to identify any differences in the simulation outputs across different runs of the same scenario.

6.3. Test results

While executing a scenario focused on an interaction between a vehicle and a cyclist during a right-turn manoeuvre, we observed inconsistencies in the cyclist's behavior across multiple runs. This inconsistency in behavior led to different simulation outcomes, prompting a comprehensive examination.

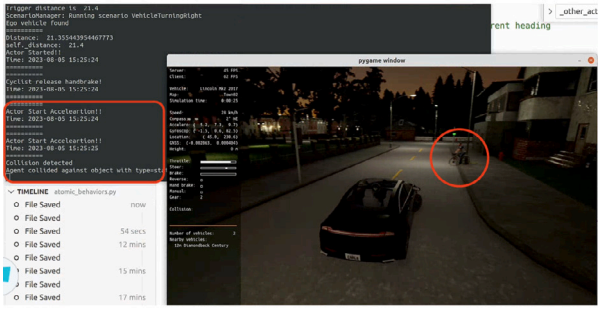
The scenario simulates a simple object collision between the ego vehicle and a cyclist. According to the predefined scenario parameters, the ego vehicle turns on autopilot mode and navigates through a road until encountering a cyclist after a right turn. The cyclist is positioned to await the entry of the ego vehicle into a certain region. Once this condition is met, the cyclist begins crossing the road. The objective is for the ego vehicle to successfully avoid a collision following the right turn, subsequently resuming its journey after the road is clear.



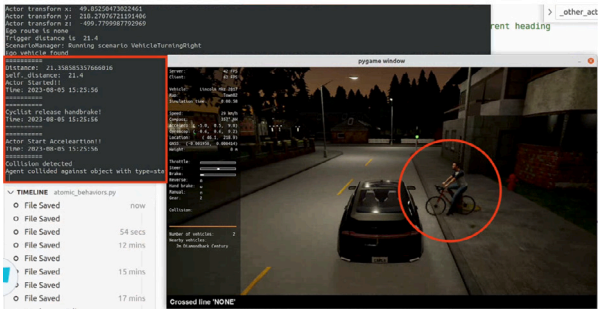
(a) The source scenario where the cyclist triggered correctly



(b) The follow-up scenario where the cyclist triggered abnormally



(c) Under normal circumstances, when the acceleration start message is sent, the cyclist will start to cross the road



(d) In the abnormal scenarios, the cyclist experienced a clear delay to start acceleration when the message is sent

Fig. 11. The cyclist's trigger timings differ significantly between the source and follow-up scenarios.

During multiple executions of the scenario, a discrepancy was observed in the cyclist's initial behaviors. The issue relates to the cyclist's acceleration phase, which is triggered when the distance between the cyclist and the ego vehicle reaches a predefined threshold. In

Table 1
Test functions and issues revealed.

Function under test	Issues revealed
The actor spawn function	The spawn function failed to prevent the actor from being created at the collision location.
Traffic-light compliance mechanisms in autopilot mode	Autopilot mode incorrectly allowed vehicles to continue after partially crossing stop lines, resulting in red-light violations.
Built-in evaluation criteria	The `CheckMaximumVelocity` criterion used incorrect speed values for assessment, leading to inaccurate evaluation results.
Consistency of simulation results	Simulation latency caused inconsistent actor behaviors, resulting in unexpected collisions.

the first iteration of the scenario, the cyclist immediately accelerated upon the trigger condition being satisfied—a behavior that aligns with expectations. The simulation had the ego vehicle wait until the cyclist had completed the crossing, and then continued driving (Fig. 11(a)). However, in other iterations, the cyclist exhibited an obvious delay of one to two seconds in acceleration after the condition was satisfied. In such circumstances, the ego vehicle could not stop in time to avoid the collision (Fig. 11(b)).

An in-depth analysis of the open-source *scenario_runner* codebase led to an examination of the *KeepVelocity* class. According to the specification [42], this class includes a function to maintain the specified velocity: The controlled actors should accelerate as quickly as possible until achieving the target velocity, which would then be maintained for the duration of the behavior. To record and compare the starting time of the cyclist's acceleration process, we introduced supplementary code into the *KeepVelocity* class. This added code was designed to log the precise timing of when the cyclist started the acceleration.

The data revealed a delay between the output of the "Start Acceleration" message in the terminal and the actual execution of the cyclist's road-crossing action. This delay is clearly illustrated in the simulation window, providing evidence of the latency (Figs. 11(c) and 11(d)). It was hypothesized that this latency could be due to computational overhead within the *KeepVelocity* class or its related components, particularly when the scenario was executed iteratively, as the issue did not arise when the scenario was executed in isolation. Additionally, the *scenario_runner* might contain timing inconsistencies or race conditions that become more pronounced during repeated executions. The variability in system load during continuous runs can also influence the simulation's performance, with higher loads causing greater inconsistencies in the behavior of traffic participants [44]. To address this issue, simulator developers should focus on identifying bottlenecks or inefficiencies in the *KeepVelocity* class to optimize the algorithms. This could involve using performance-analysis tools to identify areas where the code is consuming excessive computational resources or causing delays.

7. Discussion

7.1. Effectiveness of MT for testing the CARLA simulator

In this study, MT has demonstrated effectiveness in assessing the reliability and performance of the CARLA simulator. The integration of MRPs into this testing framework has been valuable, serving to accelerate and simplify the process of generating MRs. This feature of configuring a GPT is especially beneficial for individuals new to MT and the CARLA simulator, providing a structured guide that facilitates effective testing even for those with limited familiarity with the system and MT.

The MRs in this study led to the rapid discovery of four significant and diverse issues in the simulator, each of which includes different effects on the accuracy of the simulation. For example, the last issue associated with latency in code execution revealed a potential bottleneck that could reduce the simulator's efficiency and precision in the continuous execution of scenarios. Table 1 summarizes the issues revealed in this study, which could have easily remained hidden if using conventional testing methods: This reinforces the role that MT – and specifically MRPs – plays in deeply testing the simulator's functionality. For instance, in the testing of $\boldsymbol{MR}_{\boldsymbol{CollisionAvoidance}}$ for the actor spawn function, traditional software tests with fixed oracles might conclude their testing after verifying that the spawn function behaves correctly after a few test cases: For example, preventing the instantiation of a second actor at the same location as the first actor, as described in the source scenario of the MT. However, the MT approach went further by slightly altering the location of the second actor to find the potential point where the two actors collide, and using the same approach to spawn a third actor to prove the existence of the error. Through the application of MT, researchers can methodically uncover inconsistencies or unexpected behaviors arising from complex interactions among the simulator's components. This, in turn, contributes to improving the overall reliability and performance of the simulator.

In summary, the application of MRPs within MT not only helps with the identification of issues, but also significantly increases the derived MRs' ability to expose a broad range of flaws. The variety and importance of the issues uncovered serve as a testament to the efficacy of MT, further enhanced by the optimized MR-generation approach provided by MRPs. This ensures a more comprehensive and efficient evaluation, making it a robust tool for uncovering vulnerabilities and improving system reliability.

7.2. Threats to validity

While the findings presented in this study offer valuable insights into the effectiveness of MT for testing the CARLA simulator, we also acknowledge certain potential threats to the validity of the results. These threats stem from factors that could potentially impact the generalizability and reliability of the outcomes.

1. MR Selection: The selection of MRs in this study may have shaped the kinds of issues that were uncovered. Due to time and resource constraints, we focused on specific, limited aspects of the simulator for testing. Although the MRs chosen have facilitated the identification of issues in various aspects of the simulator, exploration of alternative relations could potentially yield further insights into the simulator's behavior.
2. Simulator Configuration and Versions: The issues identified could be influenced by the specific configuration and version of the CARLA simulator used in the study. For example, such issues may manifest within particular versions of the software or may be obscured or mitigated by updates to other components. Different configurations or simulator versions might yield varying results.
3. Test Scenario Representation: The test scenarios used in this study could not encompass the full spectrum of real-world driving situations. Variations in scenarios and environmental conditions could lead to different issue-discovery rates.

Table 2
Evaluation of SUT types across different metrics.

SUT Types	Correctness	Applicability	Novelty	Clarity	Relevance to SUT	Overall usefulness	Computational feasibility	Average
Search Engine	4.1	4.3	2.4	3.7	4.4	3.3	4.6	3.8
Navigation Software	3.6	4.3	2.8	4.0	4.7	3.8	4.0	3.9
Email App	3.8	3.8	2.1	3.5	4.3	3.0	4.4	3.6
Ordering System	3.6	4.2	2.6	3.8	3.7	2.8	4.6	3.6

4. Human Bias: Human involvement in selecting the MRs and interpreting the results could introduce biases that impact on the objectivity of the study.
5. Reproducibility of Results: The evolving nature and stochasticity of LLMs might affect the reproducibility of the results [45]. However, the proposed configuration in this study standardizes the response mechanism without modifying the underlying training data. As models are further trained with expanded datasets, their accuracy is expected to improve, potentially enhancing the quality of the generated MRs.

7.3. Performance of the GPT-MR generator

7.3.1. Performance in this study

In our previous research [36], we found that ChatGPT could generate useful MRs, but the quality and correctness were not impressive. Specifically, not every MR generated in the model's initial response was correct or effective. This performance may be attributed to the quality of training data and the ability of the model (GPT-3.5 [46]). However, with the adoption of GPT-4 in this study (the base model behind the *GPTs* feature by OpenAI, and the current most powerful ChatGPT version [34]), and the configurations involving the MRPs and guidelines, the MRs derived by the GPT-MR generator have shown obvious improvements in quality and clarity. Specifically, the MRs were correctly derived following the MRPs; they could be applied directly to testing; and some have led to the discovery of defects. Additionally, the situation where users needed to provide multiple prompts to improve the quality of MRs in the previous study did not occur with the GPT-MR generator: The configurations limited the search space of the customized GPT and improved the accuracy of the answers, allowing the GPT to provide responses that could be directly adopted [37].

However, we also found that the diversity of MRs was limited (Tables B.5 to B.8 in Appendix B). For instance, among the MRs derived by the model for different functions under test, there was always one MR that included increasing the traffic density/complexity of the scenario. In other words, the innovative features of the answers were limited, which is a common issue with GPT-generated responses [46]. Additionally, the selection and refinement of the MRs derived by the GPT still required users to possess a basic knowledge of MT, even though this knowledge could be easily obtained through user interactions with the GPT.

7.3.2. Applying the GPT-MR generator to derive MRs in other domains

Although the GPT-MR generator was initially designed for ADS-simulator testing, it can easily be applied to derive MRs for other domains. We conducted a short study where the model was asked to derive MRs for SUTs from different domains. The SUTs selected were search engines and navigation software, both commonly used and well-represented in the MT literature [47,48]. To evaluate the produced MRs, we applied criteria from our earlier study that assessed the quality of MRs produced by ChatGPT [36]. To minimize bias from existing MRs, generalized prompts like "I want to create MRs for a search engine" were used, ensuring the model produced MRs without any potential influence from its previous training. Additionally, to verify the model's ability to "infer" rather than "retrieve" MRs from the database [35], it was tasked with generating MRs for an email program and an ordering system, which represent SUTs that are commonly used in everyday life and had not been tested by MT in the literature.

Four experienced MT evaluators from diverse fields assessed the MRs: Their average scores are presented in Table 2. The results show that all MRs scored over three out of five in correctness, applicability, clarity, relevance, and computational feasibility, indicating their general accuracy and utility. However, novelty and overall usefulness scored slightly lower, consistent with the previous findings on ChatGPT's MR generation [36].

Some MRs for SUTs tested with MT, like search engines and navigation software, showed similarities to existing literature. For example, both the MRs generated for the search engine and those found in the literature involved reordering query terms [47], while both MRs for navigation software involved minor positional shifts in starting and destination points that would leave the route unchanged [48], showing the generator's ability to produce human-comparable MRs. These MRs are detailed in Table B.3 in Appendix B.

For SUTs like the email app and ordering system that had not been tested by MT in the literature, the generated MRs were similarly valuable, with clear input–output relations and effective rationales, as shown in Table B.4 in Appendix B. These findings highlight the ability of the GPT-MR generator to produce MRs across different domains.

This short study shows that, by using simple prompts (such as "derive MRs for a search engine" or "generate MRs for an email program"), the GPT-MR generator can effectively produce MRs with clear input–output relations for both MT-tested and untested systems across different domains. This suggests that the proposed human-AI hybrid MT framework can help researchers and practitioners to quickly generate MRs for unfamiliar systems, perhaps that they are testing for the first time.

Building on this evidence, the primary goal of this paper is to introduce the framework and the customized ChatGPT model, demonstrating its capability in generating effective MRs, even with simple prompts. Due to the in-context learning feature of LLMs [35], the richer and more structured the prompts that the user gives are, the more accurate its responses tend to be. Additionally, this model is configured using MRPs to narrow the answer-generation scope, and to improve the accuracy in MR generation. It does this while maintaining its capabilities in other tasks. Therefore, when interacting with this tool, users need only provide information about the SUT, and then allow the model to generate MRs. They can then either use these MRs directly, or further refine them for certain aspects (such as clarity or novelty), following the frameworks and methodologies outlined in this study. This would be especially helpful for users who are unfamiliar with the SUT, or brainstorming ideas from the answers given by the model, which is a common motivation to use LLMs [49].

7.3.3. Conclusion

In summary, using the capabilities of GPT-4, along with the configurations of MRPs and guidelines, has enhanced the quality and

applicability of MRs derived by the GPT-MR generator. Experiments have demonstrated that this generator can produce MRs across multiple domains. However, the results showed limited variation and innovation. This indicates that while the GPT-MR generator has improved the MR generation, there is still potential for further improvements in order to boost the diversity and creativity of its responses.

Furthermore, although the GPT-MR generator could effectively generate MRs for AD-simulator testing, the use of generative AI in safety-critical applications still faces challenges [50]. For instance, the tool may be sensitive to input variations, and could also retain biases that exist in the training data. To address these concerns, our methodology includes expert review to ensure that the tool not only enhances efficiency, but also adheres to the safety standards required in ADS testing.

Therefore, to maximize the capabilities of the GPT-MR generator, users are advised to be familiar with MT and the SUT before using the MR generator. While the model can produce MRs with simple prompts, enriching the details of the requests would make the results more accurate and useful.

7.4. Future work

In terms of future research, several directions present promising opportunities to build upon the current study's findings. Firstly, the derived MRs could be extended to evaluate a broader range of functionalities within the CARLA simulator, thereby providing a more comprehensive evaluation of its overall reliability and performance. Furthermore, a systematic quantitative analysis comparing GPT-generated and human-derived MRs can be conducted, using metrics like fault-detection rates and coverage improvement, along with methods for identifying biases.

Beyond using the existing MRPs, there is potential for developing additional MRPs that are tailored to the specific functionalities or complexities of the simulator. These new MRPs could offer a more specialized or streamlined approach to MR generation, optimizing the testing process for distinct use cases. Additionally, these new MRPs could involve patterns of output transformations, something that the current MRPs do not possess (they focus on output-equivalence relations in the derived MRs [17]), to enhance the diversity of the MRs generated and the capabilities of the GPT-MR generator.

Thirdly, the performance of the framework can be further explored by applying it to the testing of other SUTs, which will improve its overall effectiveness. Conducting a comprehensive quantitative analysis of how the framework enhances MT efficiency would help reveal areas for improvement. Moreover, as mentioned in Section 2.2.2, the configuration used to customize the GPT-MR generator could also be used to enhance the responses of other LLMs. This would expand the framework's applicability, offering users the flexibility to select their preferred platform. Finally, optimizing the test-harness architecture to integrate advanced LLM functionalities, such as automated test-scenario generation, can further improve efficiency and reduce human effort.

8. Conclusion

In conclusion, the human-AI hybrid MT framework has proven to be effective at identifying bugs in the CARLA simulator. The customized GPT-MR generator facilitated the generation of MRs and simplified the testing process, making it more user-friendly, especially for users less acquainted with the simulator and MT. The test harness we developed further automated this process, enhancing the efficiency of generating and executing test scenarios while allowing for extensive examination of the simulator's diverse functionalities. The revealed issues underscore the importance of the evaluation of various facets of the simulator's functionality to ensure its reliability and real-world applicability. While there are acknowledged threats to validity, the insights gained from this study are valuable, highlighting both the strengths and the limitations of employing MT, enhanced by MRPs, for evaluating complex systems. Future research work will include the development of additional or alternative MRPs, refinement of existing MRs, and the expansion of the framework, all aimed at achieving an even more comprehensive and effective issue-detection process.

CRediT authorship contribution statement

Yifan Zhang: Formal analysis, Writing – original draft, Investigation, Software, Methodology. **Tsong Yueh Chen:** Writing – review & editing, Funding acquisition, Validation, Supervision. **Matthew Pike:** Funding acquisition, Writing – review & editing, Supervision. **Dave Towey:** Writing – review & editing, Conceptualization, Supervision, Funding acquisition. **Zhihao Ying:** Validation, Writing – review & editing. **Zhi Quan Zhou:** Writing – review & editing, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was in part by the Australian Research Council Project under Grant DP210102447. The authors also acknowledge financial support from the AIOP and SSNI research groups, and the Faculty of Science and Engineering, University of Nottingham Ningbo China.

Appendix A. Bug reports and chat history links

- Bug in spawn function: https://github.com/carla-simulator/carla/issues/6653
- Bug in the traffic light detect function: https://github.com/carla-simulator/carla/issues/6684
- Bug in the built-in criteria: https://github.com/carla-simulator/scenario_runner/issues/1009
- Bug in the simulation consistency: https://github.com/carla-simulator/scenario_runner/issues/1010
- MRs derived for search engine: https://chatgpt.com/share/66f0fc4d-5b74-800e-9bf1-9736ec75b7af
- MRs derived for navigation software: https://chatgpt.com/share/66f29a81-8fe4-800e-8863-678a0e19cd83
- MRs derived for an email app: https://chatgpt.com/share/66f27670-ca64-800e-ad8d-da81e87690b1
- MRs derived for an ordering system: https://chatgpt.com/share/66f2a6e1-8178-800e-a056-c0ebef158ef7

Appendix B. MRs derived by the GPT-MR generator

See Tables B.3–B.8.

Data availability

Data will be made available on request.

Table B.3
Similar MRs derived by the model and those in the literature.

SUTs	MRs derived by the model	MRs in the literature
Search Engine	**MR: Slight reorder of query terms** **Source Input:** Query: "electric car prices" **Relation:** Reorder query: "car electric prices". The search results should remain unchanged, as the reordering does not alter the intended meaning of the query.	**MR: SwapJD** [47] **Description:** The source query *A* contains only two words (without quotation marks) and the follow-up query *B* is constructed by swapping the two words. A stable search engine should return similar results for *A* and *B* if these two queries have similar meanings, regardless of their word orders.
Navigation Software	**MR: Minor Position Shift** **Source Input:** The navigation software generates a route from Point A to Point B. Relation: Slightly shift the starting point by a few meters (e.g., 5 meters in any direction). The system should still generate the same route, as this minor change should not influence the overall route. This MR verifies that the system can handle small GPS inaccuracies without deviating from the expected route.	**MR: MRSimilar** [48] **Description:** Slight variations in starting or ending points should not result in drastically different route recommendations.

Table B.4
Example of valuable MRs from SUTs not tested with MT.

SUTs	MR Description
Email App	**MR: Small Change in Email Subject** **Scenario:** The user sends an email with the subject line "Meeting at 10 AM". **Modification (Marginal Revision):** The subject is changed to "Meeting at 10 A.M." (minor punctuation revision). **Expected Behavior:** The email should still be sent successfully, and the system should not treat the revised subject as significantly different in functionality. **Rationale:** Ensures that minor textual changes do not affect the ability to send or classify the email.
Ordering System	**MR: Amplify Order Quantity** **Original Scenario:** Place an order with 10 units of an item. **Amplified Scenario:** Increase the order to 100 units of the same item. **Expected Outcome:** The system should show an amplified increase in total price and inventory update. If inventory is insufficient, the system should prompt a warning or partial order. **Rationale:** This MR ensures that changes in order quantity lead to proportional or consistent changes in total cost and inventory levels, helping verify if the system appropriately handles large orders.

Table B.5
MRs derived for testing the spawn function.

MRP	MR	Source scenario	Follow-up scenario	Expected outcome	Rationale
$MRP_{ImpactAmplification}$	$MR_{AmplifiedActorDensity}$	Multiple actors are spawned in a region with no initial collisions.	Increase the density by spawning additional actors in the same region.	The simulator should prevent spawning in locations that result in collisions. Adjust spawn points to avoid collisions.	Tests simulator's ability to handle higher densities and ensure collision-free spawning.
	$MR_{AmplifiedComplexity}$	Actors are spawned around static obstacles without causing collisions.	Increase complexity by adding more static obstacles near the original spawn points.	The simulator should continue to prevent collisions despite increased obstacle density.	Ensures the spawn function can handle more complex environments and still avoid collisions.
$MRP_{MarginalEnsembleRevision}$	$MR_{SlightPositionalShifts}$	Actors are spawned without causing collisions.	Slightly shift spawn points by small margins.	The system should maintain collision-free spawning despite minor positional changes.	Tests precision and robustness of the spawn function with small adjustments.
	$MR_{MinorOrientationChanges}$	Actors are spawned without causing collisions.	Adjust actor orientations by small angles.	The simulator should prevent collisions as minor orientation changes should not affect collision avoidance.	Verifies consistent collision avoidance despite minor orientation changes.

Table B.6
MRs derived for testing the autopilot function.

MRP	MR	Source Scenario	Follow-up Scenario	Expected Outcome	Rationale
$MRP_{ImpactAmplification}$	$MR_{AmplifiedTrafficDensity}$	The ego vehicle approaches a junction with a red traffic light, with light traffic (e.g., a few vehicles) in the vicinity.	Increase the traffic density by adding more vehicles around the junction.	The ego vehicle should still recognize the red traffic light and stop, regardless of the increased traffic density.	Tests the Autopilot's capability to handle increased traffic complexity while maintaining compliance with traffic signals.
	$MR_{AmplifiedTrafficSignalComplexity}$	The ego vehicle approaches a simple junction with a standard red traffic light.	Introduce additional traffic control signals (e.g., pedestrian crossings with their own signals, or additional lanes with separate signals) at the junction.	The ego vehicle should correctly interpret the red traffic light for its lane and stop, despite the increased complexity of traffic signals.	Ensures the Autopilot can handle more complex traffic signal scenarios without misinterpreting signals.
$MRP_{MarginalEnsembleRevision}$	$MR_{SlightPositionalShifts}$	The ego vehicle is positioned at a specific distance from the junction with a red traffic light.	Slightly adjust the starting position of the ego vehicle (e.g., a few meters forward or backward).	The ego vehicle should still recognize the red traffic light and stop, regardless of its slightly altered starting position.	Tests the robustness of the Autopilot in maintaining correct behavior with minor positional variations.
	$MR_{MinorTimingAdjustments}$	The ego vehicle approaches the junction as the traffic light is red.	Introduce slight timing variations to the traffic light cycle (e.g., the red light duration is slightly extended or shortened).	The ego vehicle should continue to comply with the traffic light, stopping when it is red, despite minor timing changes.	Verifies that the Autopilot consistently reacts appropriately to red traffic lights even with small changes in the signal timing.
	$MR_{MinorEnvironmentalChanges}$	The ego vehicle approaches a junction with a red traffic light under standard environmental conditions.	Introduce minor environmental changes such as slight variations in weather (e.g., light rain or fog) or time of day (e.g., dawn or dusk).	The ego vehicle should still recognize and stop for the red traffic light, regardless of these minor environmental changes.	Tests the Autopilot's ability to handle slight environmental variations while maintaining correct behavior at traffic signals.

Table B.7
MRs derived for testing the simulator evaluation criteria.

MRP	MR	Source Scenario	Follow-up Scenario	Expected Outcome	Rationale
$MRP_{ImpactAmplification}$	$MR_{AmplifiedTrafficComplexity}$	A basic scenario where the ego vehicle completes a route with minimal traffic and passes through checkpoints without incidents.	Increase the traffic complexity by adding more vehicles, pedestrians, and obstacles along the route.	The evaluation criteria should accurately assess whether the scenario was completed successfully or failed, considering the increased traffic complexity. Successful completion should still be recognized if the ego vehicle navigates through without collisions or rule violations.	Tests the criteria's ability to handle and accurately evaluate more complex traffic scenarios.
	$MR_{AmplifiedEnvironmentalChallenges}$	A scenario where the ego vehicle drives through clear weather conditions and straightforward road conditions.	Introduce challenging environmental conditions such as heavy rain, fog, or night-time driving.	The evaluation criteria should still correctly determine if the scenario was completed successfully or failed, considering the amplified environmental challenges. The criteria should recognize successful completion if the ego vehicle adapts to the challenging conditions without incidents.	Ensures the evaluation criteria are robust and can accurately assess performance under different environmental conditions.
$MRP_{MarginalEnsembleRevision}$	$MR_{SlightPositionalAdjustments}$	A scenario where the ego vehicle starts and ends at specific checkpoints and completes the route without incidents.	Slightly adjust the starting and ending positions of the ego vehicle by a few meters.	The evaluation criteria should still correctly assess the scenario as successful or failed, based on the ego vehicle's performance, despite the slight positional adjustments.	Tests the evaluation criteria's ability to handle minor positional changes without affecting the assessment of scenario success.
	$MR_{MinorRouteVariations}$	A scenario where the ego vehicle follows a predefined route and passes all checkpoints successfully.	Introduce minor variations in the route, such as slight detours or alternative paths that still lead to the same checkpoints.	The evaluation criteria should recognize the scenario as successful if the ego vehicle follows the varied route correctly and reaches all checkpoints without incidents.	Verifies that the evaluation criteria are flexible and can accurately assess different but valid route variations.
	$MR_{SubtleBehavioralChanges}$	A scenario where the ego vehicle drives at a constant speed and adheres to all traffic rules, completing the route successfully.	Introduce subtle changes in the ego vehicle's behavior, such as slight variations in speed or minor adjustments in lane positioning, without violating traffic rules.	The evaluation criteria should still assess the scenario as successful if the ego vehicle completes the route correctly, despite the subtle behavioral changes.	Ensures the evaluation criteria are robust and can accurately evaluate the ego vehicle's performance even with minor behavioral variations.

Table B.8
MRs derived for testing the consistency of simulation outputs.

MRP	MR	Source Scenario	Follow-up Scenario	Expected Outcome	Rationale
$MRP_{ImpactAmplification}$	$MR_{RepeatedRunsWithAmplifiedFactors}$	A baseline scenario where the ego vehicle drives through a predefined route with standard traffic and environmental conditions.	Increase the number of runs for the same scenario under identical conditions (e.g., 10 repeated runs).	The simulation outputs should be consistent across all runs, showing similar ego vehicle behaviors, routes, and interactions with other entities.	Tests the simulator's ability to produce consistent outputs under repeated runs with amplified repetition factors.
$MRP_{MarginalEnsembleRevision}$	$MR_{SlightVariationsInInitialConditions}$	A scenario where the ego vehicle starts from a specific position and drives through a route with predefined traffic and environmental conditions.	Introduce slight variations in the initial conditions, such as small changes in the starting position (e.g., a few centimetres) or slight timing differences (e.g., starting the scenario a few milliseconds later).	The simulation outputs should remain consistent, with the ego vehicle exhibiting similar behaviors and outcomes despite the slight initial variations.	Tests the robustness of the simulator in maintaining consistent outputs with minor changes in initial conditions.
	$MR_{MinorEnvironmentalChanges}$	A scenario where the ego vehicle drives through a route with standard environmental conditions (e.g., clear weather, daylight).	Introduce minor changes in the environment, such as slight variations in weather (e.g., light fog) or time of day (e.g., early morning light).	The simulation outputs should be consistent, showing similar ego vehicle behaviors and outcomes despite the minor environmental changes.	Ensures the simulator's consistency in producing outputs under slightly different environmental conditions.
	$MR_{SmallAdjustmentsInTrafficDensity}$	A scenario where the ego vehicle drives through a route with a specific number of other vehicles and pedestrians.	Introduce small adjustments in the traffic density, such as adding or removing one vehicle or pedestrian.	The simulation outputs should remain consistent, with the ego vehicle's behavior and interactions being similar despite the small changes in traffic density.	Verifies the simulator's ability to maintain consistent outputs with minor variations in traffic conditions.
	$MR_{SubtleBehavioralTweaks}$	A scenario where the ego vehicle follows a specific set of driving behaviors (e.g., speed, lane positioning).	Introduce subtle tweaks to the ego vehicle's behavior, such as slight variations in speed or lane positioning.	The simulation outputs should be consistent, showing similar overall behaviors and outcomes for the ego vehicle despite the subtle behavioral tweaks.	Tests the consistency of the simulator in producing outputs when there are minor variations in the ego vehicle's behavior.

References

[1] C. Badue, R. Guidolini, R.V. Carneiro, P. Azevedo, V.B. Cardoso, A. Forechi, L. Jesus, R. Berriel, T.M. Paixão, F. Mutz, L. de Paula Veronese, T. Oliveira-Santos, A.F. De Souza, Self-driving cars: A survey, Expert Syst. Appl. 165 (2021) 113816, http://dx.doi.org/10.1016/j.eswa.2020.113816.

[2] C.-H. Yu, Y.-Z. Chen, I.-C. Kuo, The benefit of simulation test application on the development of autonomous driving system, in: 2020 International Automatic Control Conference, CACS, 2020, pp. 1–5, http://dx.doi.org/10.1109/CACS50047.2020.9289717.

[3] V. Calhoun, G. Pearlson, A selective review of simulated driving studies: Combining naturalistic and hybrid paradigms, analysis approaches, and future directions, NeuroImage 59 (1) (2012) 25–35, http://dx.doi.org/10.1016/j.neuroimage.2011.06.037, Neuroergonomics: The human brain in action and at work.

[4] W. Li, C.W. Pan, R. Zhang, J.P. Ren, Y. Ma, J. Fang, F. Yan, Q. Geng, X. Huang, H. Gong, W. Xu, G. Wang, D. Manocha, R. Yang, AADS: Augmented autonomous driving simulation using data-driven algorithms, Sci. Robot. 4 (28) (2019) http://dx.doi.org/10.1126/scirobotics.aaw0863.

[5] X. Bai, Y. Luo, L. Jiang, A. Gupta, P. Kaveti, H. Singh, S. Ostadabbas, Bridging the domain gap between synthetic and real-world data for autonomous driving, J. Auton. Transp. Syst. 1 (2023) 1–15.

[6] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, V. Koltun, CARLA: An open urban driving simulator, in: Proceedings of the 1st Annual Conference on Robot Learning, 2017, pp. 1–16.

[7] N.A. Stanton, On the reliability and validity of, and training in, ergonomics methods: a challenge revisited, Theor. Issues Ergon. Sci. 17 (4) (2016) 345–353.

[8] A. Geiger, P. Lenz, R. Urtasun, Are we ready for autonomous driving? The KITTI vision benchmark suite, in: 2012 IEEE Conference on Computer Vision and Pattern Recognition, 2012, pp. 3354–3361, http://dx.doi.org/10.1109/CVPR.2012.6248074.

[9] E.T. Barr, M. Harman, P. McMinn, M. Shahbaz, S. Yoo, The oracle problem in software testing: A survey, IEEE Trans. Softw. Eng. 41 (5) (2015) 507–525.

[10] Z.Q. Zhou, L. Sun, Metamorphic testing of driverless cars, Commun. ACM 62 (3) (2019) 61–67.

[11] R.A. Wynne, V. Beanland, P.M. Salmon, Systematic review of driving simulator validation studies, Saf. Sci. 117 (2019) 138–151.

[12] T.Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T.H. Tse, Z.Q. Zhou, Metamorphic testing: A review of challenges and opportunities, ACM Comput. Surv. 51 (1) (2018) 4:1–4:27.

[13] S. Segura, D. Towey, Z.Q. Zhou, T.Y. Chen, Metamorphic testing: Testing the untestable, IEEE Softw. 37 (3) (2020) 46–53, http://dx.doi.org/10.1109/MS.2018.2875968.

[14] Z. Ying, D. Towey, A. Bellotti, Z. Zhou, MRGS-ART: Metamorphic relation and group selection based on adaptive random testing, Softw. Test. Verif. Reliab. 35 (1) (2024) e1908, http://dx.doi.org/10.1002/stvr.1908.
[15] Q.-H. Luu, H. Liu, T.Y. Chen, H.L. Vu, A sequential metamorphic testing framework for understanding autonomous vehicle's decisions, IEEE Trans. Intell. Veh. (2024).
[16] Y. Zhang, D. Towey, M. Pike, J.C. Han, G. Zhou, C. Yin, Q. Wang, C. Xie, Metamorphic testing harness for the baidu apollo perception-camera module, in: 2023 IEEE/ACM 8th International Workshop on Metamorphic Testing, MET, IEEE, 2023, pp. 9–16.
[17] Y. Zhang, D. Towey, M. Pike, J.C. Han, Z.Q. Zhou, C. Yin, Q. Wang, C. Xie, Scenario-driven metamorphic testing for autonomous driving simulators, Softw. Test. Verif. Reliab. 34 (7) (2024) e1892, http://dx.doi.org/10.1002/stvr.1892.
[18] M. Iqbal, J.C. Han, Z.Q. Zhou, D. Towey, T.Y. Chen, Metamorphic testing of advanced driver-assistance system (ADAS) simulation platforms: Lane keeping assist system (LKAS) case studies, Inf. Softw. Technol. 155 (2023) 107104.
[19] Z. Ying, D. Towey, A. Bellotti, T.Y. Chen, Z. Zhou, SFIDMT-ART: A metamorphic group generation method based on adaptive random testing applied to source and follow-up input domains, Inf. Softw. Technol. 175 (2024) 107528, http://dx.doi.org/10.1016/j.infsof.2024.107528.
[20] R. Li, H. Liu, P.-L. Poon, D. Towey, C.-A. Sun, Z. Zheng, Z.Q. Zhou, T.Y. Chen, Metamorphic relation generation: State of the art and visions for future research, 2023, arXiv preprint arXiv:2406.05397. Available: https://arxiv.org/abs/2406.05397.
[21] Z.Q. Zhou, L. Sun, T.Y. Chen, D. Towey, Metamorphic relations for enhancing system understanding and use, IEEE Trans. Softw. Eng. 46 (10) (2020) 1120–1154, http://dx.doi.org/10.1109/TSE.2018.2876433.
[22] S. Segura, J.A. Parejo, J. Troya, A. Ruiz-Cortés, Metamorphic testing of RESTful web APIs, IEEE Trans. Softw. Eng. 44 (11) (2018) 1083–1099, http://dx.doi.org/10.1109/TSE.2017.2764464.
[23] S. Stević, M. Krunić, M. Dragojević, N. Kaprocki, Development and validation of ADAS perception application in ROS environment integrated with CARLA simulator, in: 2019 27th Telecommunications Forum, TELFOR, IEEE, 2019, pp. 1–4.
[24] E. Zapridou, E. Bartocci, P. Katsaros, Runtime verification of autonomous driving systems in CARLA, in: International Conference on Runtime Verification, Springer, 2020, pp. 172–183.
[25] S. Ramakrishna, B. Luo, C.B. Kuhn, G. Karsai, A. Dubey, Anti-CARLA: An adversarial testing framework for autonomous vehicles in CARLA, in: 2022 IEEE 25th International Conference on Intelligent Transportation Systems, ITSC, IEEE, 2022, pp. 2620–2627.
[26] P. Kaur, S. Taghavi, Z. Tian, W. Shi, A survey on simulators for testing self-driving cars, in: 2021 Fourth International Conference on Connected and Autonomous Driving (MetroCAD), IEEE, 2021, pp. 62–70.
[27] A. Salah, R. Dssouli, G. Lapalme, Compiling real-time scenarios into a timed automaton, in: Formal Techniques for Networked and Distributed Systems, Springer US, Boston, MA, 2001, pp. 135–150.
[28] CARLA Simulator, Core actors documentation, 2023, (Accessed 4 June 2023). Available: https://carla.readthedocs.io/en/docs-preview/core_actors/.
[29] CARLA Simulator, Advanced traffic manager documentation, 2023, (Accessed 4 June 2023). Available: https://carla.readthedocs.io/en/latest/adv_traffic_manager/.
[30] T. Menzel, G. Bagschik, L. Isensee, A. Schomburg, M. Maurer, From functional to logical scenarios: Detailing a keyword-based scenario description for execution in a simulation environment, in: 2019 IEEE Intelligent Vehicles Symposium, IV, IEEE, 2019, pp. 2383–2390.
[31] I. Verma, D. Kumar, R. Goel, Implementation and comparison of artificial intelligence techniques in software testing, in: 2023 6th International Conference on Information Systems and Computer Networks, ISCON, 2023, pp. 1–9, http://dx.doi.org/10.1109/ISCON57294.2023.10112041.
[32] M. Islam, F. Khan, S. Alam, M. Hasan, Artificial intelligence in software testing: A systematic review, in: TENCON 2023 - 2023 IEEE Region 10 Conference, TENCON, 2023, pp. 524–529, http://dx.doi.org/10.1109/TENCON58879.2023.10322349.
[33] C. Tsigkanos, P. Rani, S. Müller, T. Kehrer, Large language models: The next frontier for variable discovery within metamorphic testing? in: 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER, 2023, pp. 678–682, http://dx.doi.org/10.1109/SANER56733.2023.00070.
[34] J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, S. Zhong, B. Yin, X. Hu, Harnessing the power of LLMs in practice: A survey on ChatGPT and beyond, ACM Trans. Knowl. Discov. from Data 18 (6) (2024) 1–32.
[35] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, Q. Wang, Software testing with large language models: Survey, landscape, and vision, IEEE Trans. Softw. Eng. 50 (4) (2024) 911–936, http://dx.doi.org/10.1109/TSE.2024.3368208.
[36] Y. Zhang, D. Towey, M. Pike, Automated metamorphic-relation generation with ChatGPT: An experience report, in: 2023 IEEE 47th Annual Computers, Software, and Applications Conference, COMPSAC, 2023, pp. 1780–1785, http://dx.doi.org/10.1109/COMPSAC57700.2023.00275.
[37] OpenAI, Introducing GPTs, 2023, (Accessed 30 July 2023). Available: https://openai.com/index/introducing-gpts/.
[38] Association for Standardization of Automation and Measuring Systems, ASAM OpenSCENARIO: User guide, 2023, (Accessed 2 June 2023). Available: https://asam.net/index.php?eID=dumpFile&t=f&f=4092&token=d3b6a55e911b22179e3c0895fe2caae8f5492467#_foreword.
[39] Y. Pan, H. Ao, Y. Fan, Metamorphic testing for autonomous driving systems in fog based on quantitative measurement, in: 2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C), 2021, pp. 30–37, http://dx.doi.org/10.1109/QRS-C55045.2021.00015.
[40] E. Yurtsever, J. Lambert, A. Carballo, K. Takeda, A survey of autonomous driving: Common practices and emerging technologies, IEEE Access 8 (2020) 58443–58469.
[41] CARLA Simulator, Scenario runner: Atomic criteria python file, 2023, (Accessed 7 June 2023). Available: https://github.com/carla-simulator/scenario_runner/blob/f1a94516db57612f22d5c42e597e350e781b9224/srunner/scenariomanager/scenarioatomics/atomic_criteria.py.
[42] CARLA Simulator, Scenario runner documentation, 2023, (Accessed 7 June 2023). Available: https://github.com/carla-simulator/scenario_runner/blob/master/Docs/getting_scenariorunner.md.
[43] C. Troost, R. Huber, A.R. Bell, H. van Delden, T. Filatova, Q.B. Le, M. Lippe, L. Niamir, J.G. Polhill, Z. Sun, et al., How to keep it adequate: A protocol for ensuring validity in agent-based simulation, Environ. Model. Softw. 159 (2023) 105559.
[44] T. Lejmi, I. Sabuncuoglu, Effect of load, processing time and due date variation on the effectiveness of scheduling rules, Int. J. Prod. Res. 40 (4) (2002) 945–974.
[45] L. Chen, M. Zaharia, J. Zou, How is ChatGPT's behavior changing over time? Harv. Data Sci. Rev. 6 (2) (2024).
[46] W. Yeadon, A. Peach, C. Testrow, A comparison of human, GPT-3.5, and GPT-4 performance in a university-level coding course, Sci. Rep. 14 (1) (2024) 23285.
[47] Z.Q. Zhou, S. Xiang, T.Y. Chen, Metamorphic testing for software quality assessment: A study of search engines, IEEE Trans. Softw. Eng. 42 (3) (2015) 264–284.
[48] J. Brown, Z.Q. Zhou, Y.-W. Chow, Metamorphic testing of navigation software: A pilot study with google maps, in: Proceedings of the 51st Hawaii International Conference on System Sciences, 2018, pp. 5687–5696, http://dx.doi.org/10.24251/HICSS.2018.713.
[49] O. Shaer, A. Cooper, O. Mokryn, A.L. Kun, H. Ben Shoshan, AI-augmented brainwriting: Investigating the use of LLMs in group ideation, in: Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems, 2024, pp. 1–17.
[50] J. Perez-Cerrolaza, J. Abella, M. Borg, C. Donzella, J. Cerquides, F.J. Cazorla, C. Englund, M. Tauber, G. Nikolakopoulos, J.L. Flores, Artificial intelligence for safety-critical systems in industrial and transportation domains: A survey, ACM Comput. Surv. 56 (7) (2024) http://dx.doi.org/10.1145/3626314.